**NAME:- ANKIT KUMAR**

**ENROLLMENT N0:- S25MCAG0010**

**LAB ASSIGNMENT:- 09**

1. List employee names along with the projects they are working on.

```
mysql> SELECT e.name, p.project_name
    -> FROM Employee e
    -> JOIN Works_On w ON e.emp_id = w.emp_id
    -> JOIN Project p ON w.project_id = p.project_id;
+------+-----------------+
| name | project_name    |
+------+-----------------+
| Amit | AI System       |
| John | AI System       |
| Riya | HR Portal       |
| Sara | Finance Tracker |
| Amit | Cloud Migration |
| John | Cloud Migration |
| Raj  | Cloud Migration |
+------+-----------------+
7 rows in set (0.01 sec)
```

2. Find employees who are not assigned to any project.

```
mysql> SELECT e.name
    -> FROM Employee e
    -> LEFT JOIN Works_On w ON e.emp_id = w.emp_id
    -> WHERE w.project_id IS NULL;
Empty set (0.01 sec)
```

3. Display project names along with total hours worked on each project.

```
mysql> SELECT p.project_name, SUM(w.hours_worked) AS total_hours
    -> FROM Project p
    -> JOIN Works_On w ON p.project_id = w.project_id
    -> GROUP BY p.project_name;
+-----------------+-------------+
| project_name    | total_hours |
+-----------------+-------------+
| AI System       |         270 |
| HR Portal       |          80 |
| Finance Tracker |         170 |
| Cloud Migration |         390 |
+-----------------+-------------+
4 rows in set (0.02 sec)
```

4. Find employees working on more than one project.

```
mysql> SELECT e.name, COUNT(w.project_id) AS project_count
    -> FROM Employee e
    -> JOIN Works_On w ON e.emp_id = w.emp_id
    -> GROUP BY e.emp_id, e.name
    -> HAVING COUNT(w.project_id) > 1;
+------+---------------+
| name | project_count |
+------+---------------+
| Amit |             2 |
| John |             2 |
+------+---------------+
2 rows in set (0.01 sec)
```

5. List employees whose department matches the project department they are working on.

```
mysql> SELECT e.name, p.project_name
    -> FROM Employee e
    -> JOIN Works_On w ON e.emp_id = w.emp_id
    -> JOIN Project p ON w.project_id = p.project_id
    -> WHERE e.department = p.department;
+------+----------------+
| name | project_name   |
+------+----------------+
| Amit | AI System      |
| John | AI System      |
| Riya | HR Portal      |
| Sara | Finance Tracker |
| Amit | Cloud Migration |
| John | Cloud Migration |
| Raj  | Cloud Migration |
+------+----------------+
7 rows in set (0.00 sec)
```

6. Find the average salary of each department.

```
mysql> SELECT department, AVG(salary) AS avg_salary
    -> FROM Employee
    -> GROUP BY department;
+------------+------------+
| department | avg_salary |
+------------+------------+
| IT         | 61666.6667 |
| HR         | 50000.0000 |
| Finance    | 65000.0000 |
+------------+------------+
3 rows in set (0.01 sec)
```

7. Find the highest and lowest salary in the company.

```
mysql> SELECT MAX(salary) AS highest_salary,
    ->        MIN(salary) AS lowest_salary
    -> FROM Employee;
+----------------+---------------+
| highest_salary | lowest_salary |
+----------------+---------------+
|          70000 |         50000 |
+----------------+---------------+
1 row in set (0.01 sec)
```

8. Calculate total hours worked by each employee.

```
mysql> SELECT e.name, SUM(w.hours_worked) AS total_hours
    -> FROM Employee e
    -> JOIN Works_On w ON e.emp_id = w.emp_id
    -> GROUP BY e.emp_id, e.name;
+------+-------------+
| name | total_hours |
+------+-------------+
| Amit |         220 |
| Riya |          80 |
| John |         350 |
| Sara |         170 |
| Raj  |          90 |
+------+-------------+
5 rows in set (0.00 sec)
```

9. Find the department with the highest average salary.

```
mysql> SELECT department, AVG(salary) AS avg_salary
    -> FROM Employee
    -> GROUP BY department
    -> ORDER BY avg_salary DESC
    -> LIMIT 1;
+------------+------------+
| department | avg_salary |
+------------+------------+
| Finance    | 65000.0000 |
+------------+------------+
1 row in set (0.00 sec)
```

10. Count number of employees in each department.

```
mysql> SELECT department, COUNT(*) AS total_employees
    -> FROM Employee
    -> GROUP BY department;
+------------+-----------------+
| department | total_employees |
+------------+-----------------+
| IT         |               3 |
| HR         |               1 |
| Finance    |               1 |
+------------+-----------------+
3 rows in set (0.00 sec)
```

11. Rank employees based on salary within each department.

```
mysql> SELECT name, department, salary,
    -> RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rank_in_dept
    -> FROM Employee;
+------+------------+--------+-------------+
| name | department | salary | rank_in_dept |
+------+------------+--------+-------------+
| Sara | Finance    |  65000 |           1 |
| Riya | HR         |  50000 |           1 |
| John | IT         |  70000 |           1 |
| Amit | IT         |  60000 |           2 |
| Raj  | IT         |  55000 |           3 |
+------+------------+--------+-------------+
5 rows in set (0.01 sec)
```

12. Find the second highest salary using window function.

```
mysql> SELECT salary
    -> FROM (
    ->     SELECT salary,
    ->     DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    ->     FROM Employee
    -> ) t
    -> WHERE rnk = 2;
+--------+
| salary |
+--------+
|  65000 |
+--------+
1 row in set (0.01 sec)
```

13. Calculate running total of salary ordered by join_date.

```
mysql> SELECT name, join_date, salary,
    -> SUM(salary) OVER (ORDER BY join_date) AS running_total
    -> FROM Employee;
+------+------------+--------+---------------+
| name | join_date  | salary | running_total |
+------+------------+--------+---------------+
| Sara | 2019-07-11 |  65000 |         65000 |
| John | 2020-03-20 |  70000 |        135000 |
| Amit | 2021-06-15 |  60000 |        195000 |
| Riya | 2022-01-10 |  50000 |        245000 |
| Raj  | 2023-02-01 |  55000 |        300000 |
+------+------------+--------+---------------+
5 rows in set (0.00 sec)
```

14. Find employees whose salary is above department average.

```
mysql> SELECT name, department, salary
    -> FROM Employee e
    -> WHERE salary > (
    ->     SELECT AVG(salary)
    ->     FROM Employee
    ->     WHERE department = e.department
    -> );
+------+------------+--------+
| name | department | salary |
+------+------------+--------+
| John | IT         |  70000 |
+------+------------+--------+
1 row in set (0.01 sec)
```

15. Assign row numbers to employees based on salary descending.

```
mysql> SELECT name, salary,
    -> ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num
    -> FROM Employee;
+------+--------+---------+
| name | salary | row_num |
+------+--------+---------+
| John |  70000 |       1 |
| Sara |  65000 |       2 |
| Amit |  60000 |       3 |
| Raj  |  55000 |       4 |
| Riya |  50000 |       5 |
+------+--------+---------+
5 rows in set (0.00 sec)
```

16. Find project with maximum total hours worked.

```
mysql> SELECT p.project_name, SUM(w.hours_worked) AS total_hours
    -> FROM Project p
    -> JOIN Works_On w ON p.project_id = w.project_id
    -> GROUP BY p.project_name
    -> ORDER BY total_hours DESC
    -> LIMIT 1;
+-----------------+-------------+
| project_name    | total_hours |
+-----------------+-------------+
| Cloud Migration |         390 |
+-----------------+-------------+
1 row in set (0.00 sec)
```

17. Find employees contributing more than average hours in their project.

```
mysql> SELECT w.emp_id, w.project_id, w.hours_worked
    -> FROM Works_On w
    -> WHERE w.hours_worked > (
    ->     SELECT AVG(hours_worked)
    ->     FROM Works_On
    ->     WHERE project_id = w.project_id
    -> );
+--------+------------+--------------+
| emp_id | project_id | hours_worked |
+--------+------------+--------------+
|      3 |        101 |          150 |
|      3 |        104 |          200 |
+--------+------------+--------------+
2 rows in set (0.00 sec)
```

18. Display employee name, project name, and percentage contribution (hours / total project hours).

```
mysql> SELECT e.name, p.project_name,
    -> (w.hours_worked * 100.0 /
    ->  SUM(w.hours_worked) OVER (PARTITION BY p.project_id)) AS contribution_percentage
    -> FROM Works_On w
    -> JOIN Employee e ON w.emp_id = e.emp_id
    -> JOIN Project p ON w.project_id = p.project_id;
+------+-----------------+------------------------+
| name | project_name    | contribution_percentage |
+------+-----------------+------------------------+
| Amit | AI System       |               44.44444 |
| John | AI System       |               55.55556 |
| Riya | HR Portal       |              100.00000 |
| Sara | Finance Tracker |              100.00000 |
| Amit | Cloud Migration |               25.64103 |
| John | Cloud Migration |               51.28205 |
| Raj  | Cloud Migration |               23.07692 |
+------+-----------------+------------------------+
7 rows in set (0.01 sec)
```

19. Write a procedure to increase salary of employees in a given department by 10%.

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE IncreaseSalary(IN dept_name VARCHAR(30))
    -> BEGIN
    ->     UPDATE Employee
    ->     SET salary = salary * 1.10
    ->     WHERE department = dept_name;
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
mysql> CALL IncreaseSalary('IT');
Query OK, 3 rows affected (0.02 sec)
```

20. Update salary of any employee and verify that the trigger logs the change correctly.

```
mysql> UPDATE Employee
    -> SET salary = 80000
    -> WHERE emp_id = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM Salary_Log;
+--------+--------+------------+------------+---------------------+
| log_id | emp_id | old_salary | new_salary | change_date         |
+--------+--------+------------+------------+---------------------+
|      1 |      1 |      60000 |      66000 | 2026-04-13 09:39:57 |
|      2 |      3 |      70000 |      77000 | 2026-04-13 09:39:57 |
|      3 |      5 |      55000 |      60500 | 2026-04-13 09:39:57 |
|      4 |      1 |      66000 |      80000 | 2026-04-13 09:40:43 |
+--------+--------+------------+------------+---------------------+
4 rows in set (0.00 sec)
```